

Class Overview

In Class:

- Dataflow Testing
- Unit Testing Retrospective
- Software Testing Ethics
- Half-Semester Project Discussion
- Exam Return

After Class: Office Hours**Homework:**

- 03/04: Half-Semester Project Idea

2

Upcoming

This Week (3/4) In Class:

- Dataflow Testing
- Unit Testing Retrospective
- Software Testing Ethics
- Half-Semester Project Discussion
- Exam Return

Next Week (3/11): Spring Break**Following Week (3/18):** Guest Lecture Attendance will be graded!

4

CIS 613: Dataflow Testing

5

Dataflow Testing

Variables matter, so why just look at code structure?

6

Definitions

- Given a program, P , with a set V of variables in P , and the program graph $G(P)$, Node $n \in G(P)$ is
 - a **defining node** of the variable $v \in V$, written as **DEF**(v, n), iff the value of the variable v is defined at the statement fragment corresponding to node n .
 - a **usage node** of the variable $v \in V$, written as **USE**(v, n), iff the value of the variable v is used at the statement fragment corresponding to node n .
- A usage node **USE**(v, n) is a predicate use (denoted as P-use) iff the statement n is a predicate statement; otherwise, **USE**(v, n) is a computation use (denoted C-use).

7

Types of Verification

- **Dynamic Verification** (e.g., software testing):
 - Software implementation is executed with test data.
 - Applied to a prototype or an executable version.
 - Actual vs expected outputs are compared.
- **Static Verification**:
 - Analyze and check system representations such as requirements, specifications, design, and source code.
 - Can be applied at all stages of the software process
 - Walkthrough, inspections, desk checking, correctness proofs, and automated static analyzers.

8

Types of Verification

- **Dynamic Verification** (e.g., software testing):
 - Software implementation is executed with test data.
 - Applied to a prototype or an executable version.
 - Actual vs expected outputs are compared.
- **Static Verification:**
 - Analyze and check system representations such as requirements, specifications, design, and source code.
 - Can be applied at all stages of the software process
 - Walkthrough, inspections, desk checking, correctness proofs, and automated static analyzers.

Dataflow Testing is **Static** Verification

10

Static Analysis

Typical variable use cases that can be detected with static analysis:

- A variable that is defined, but never used (referenced).
- A variable that is used before it is defined.
- A variable that is defined twice before it is used.

11

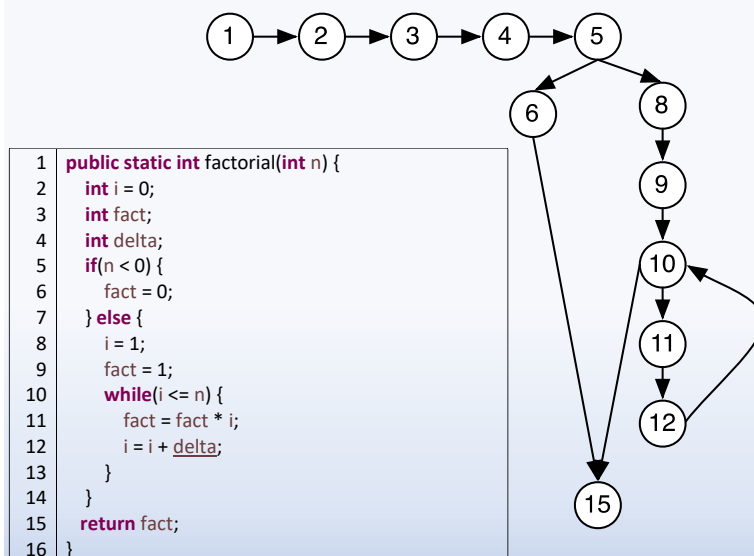
In Class Exercise

On the handout, perform the following steps:

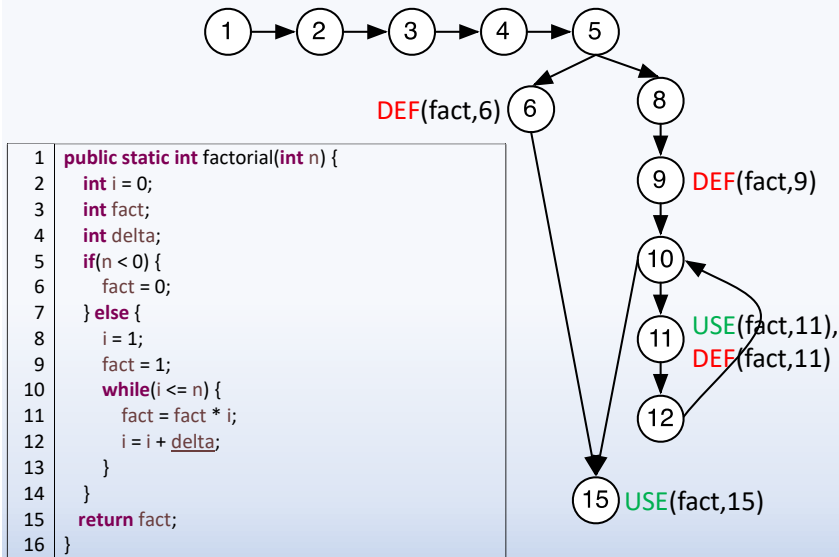
- Create a program graph for the factorial program.
- Identify the:
 - Defining Node: DEF(v,n)
 - Usage Node: USE(v,n)
- Identify instances of:
 - A variable that is defined, but never used (referenced).
 - A variable that is used before it is defined.
 - A variable that is defined twice before it is used.
- What general rules can be used to identify these instances.

12

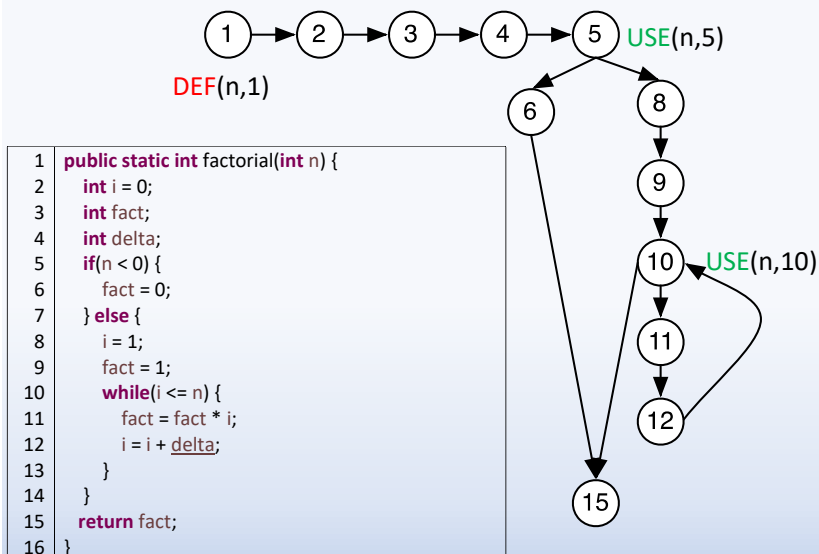
In Class Exercise



13

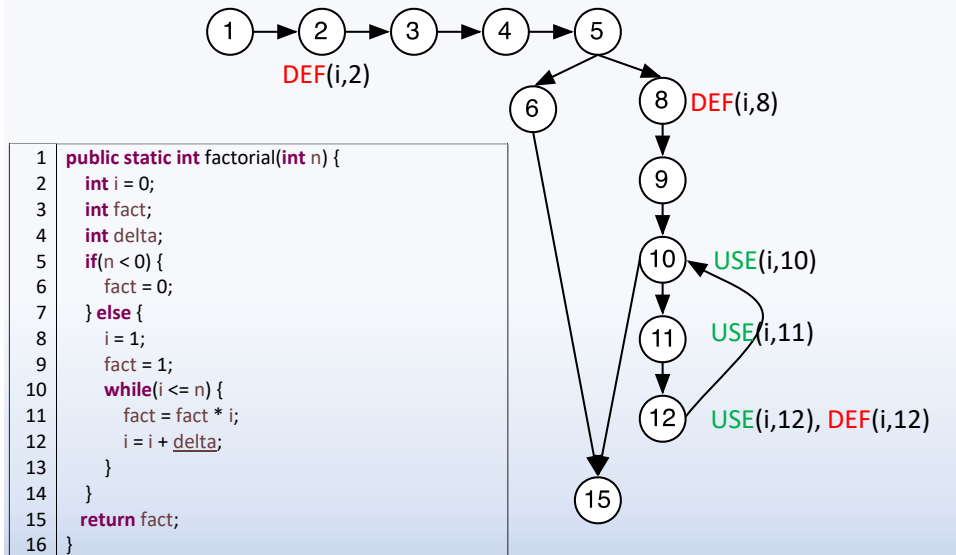
In Class Exercise: DEF, USE for *fact*

14

In Class Exercise: DEF, USE for *n*

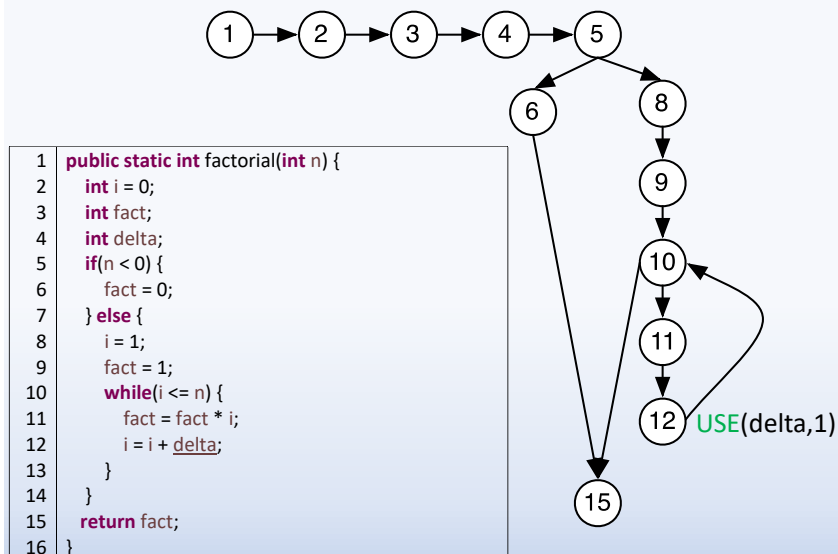
15

In Class Exercise: DEF, USE for *i*



16

In Class Exercise: DEF, USE for *delta*



17

Definitions

- A **definition-use path** with respect to a variable v (denoted **du-path**) is a path in the set of all paths in P , $PATHS(P)$, such that for some $v \in V$, there are define and usage nodes $DEF(v, m)$ and $USE(v, n)$ such that m and n are the initial and final nodes of the path.
- A **definition-clear path** with respect to a variable v (denoted **dc-path**) is a definition-use path in $PATHS(P)$ with initial and final nodes $DEF(v, m)$ and $USE(v, n)$ such that no other node in the path is a defining node of v .

18

Define/Use Test Cases

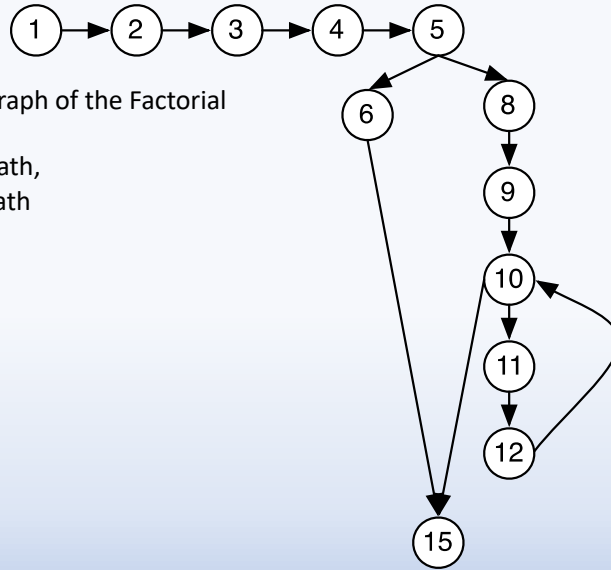
- Technique: for a particular variable,
 - find all its definition and usage nodes, then
 - find the du-paths and dc-paths among these.
 - for each path, devise a "suitable" (functional?) set of test cases.

How does the static analysis we performed on the Factorial function compare to DU- and DC-Paths?

19

In Class Assignment

- Based the program graph of the Factorial function, identify:
 - At least one DU-Path,
 - At least one DC-Path

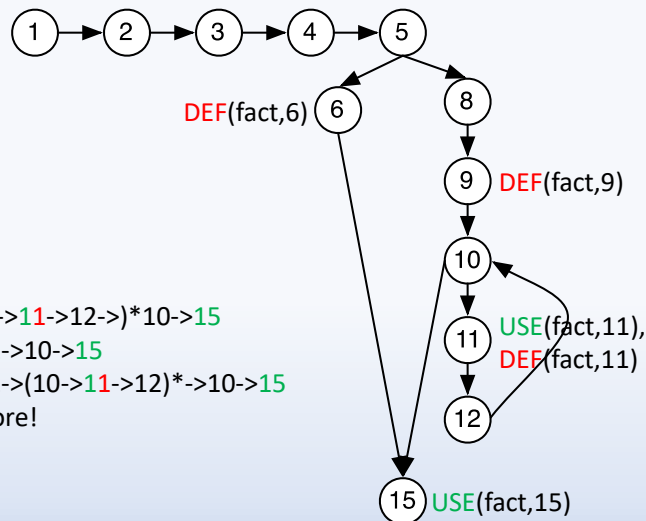


20

In Class Exercise: DEF, USE for *fact*

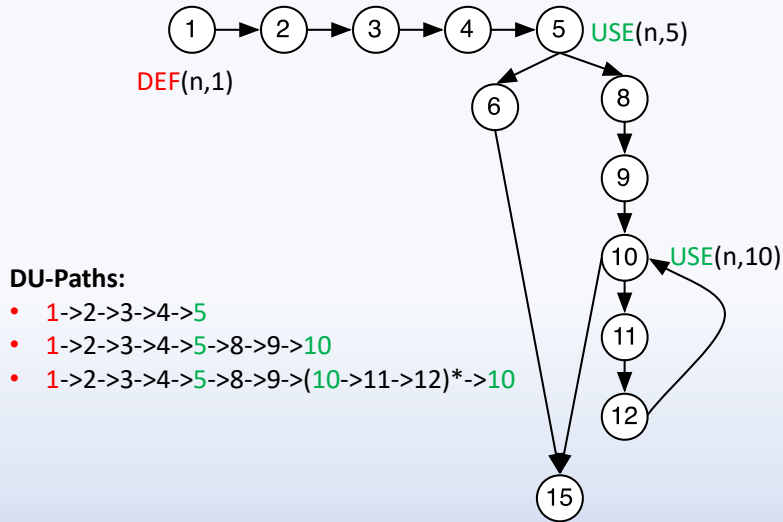
DU-Paths:

- 11
- 6->15
- 9->15
- 9->(10->11->12->)*10->15
- 11->12->10->15
- 11->12->(10->11->12)*->10->15
- And More!



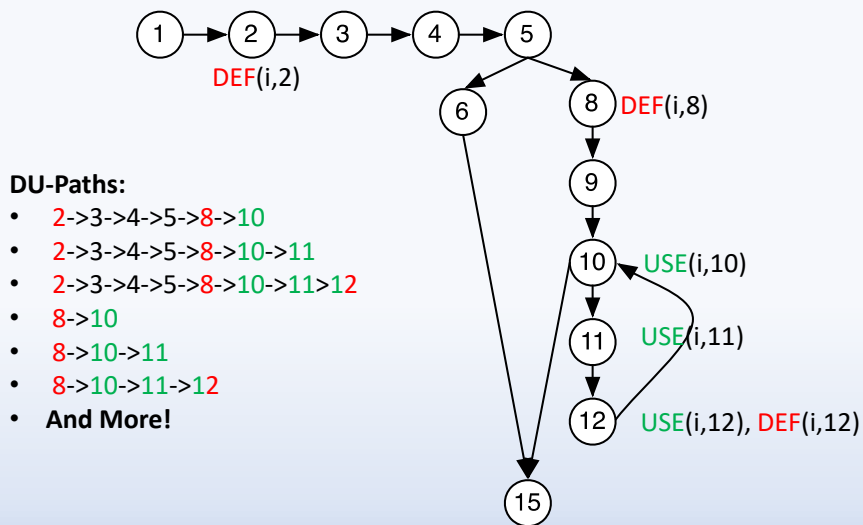
21

In Class Exercise: DEF, USE for n



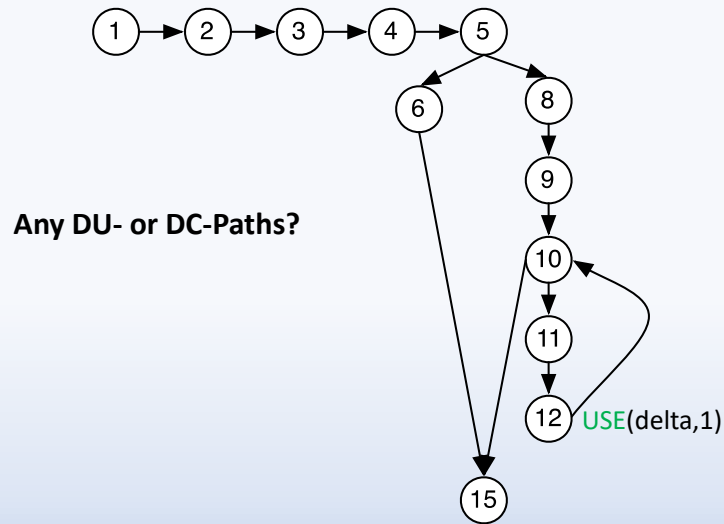
22

In Class Exercise: DEF, USE for i



23

In Class Exercise: DEF, USE for *delta*



24

Coverage Metrics

DC-Path Coverage	Graph Coverage
All Paths	All Paths
All DU-Paths	All Edges
All Uses	All Nodes

25

Dataflow Testing Strategies

- Dataflow testing is indicated in
 - Computation-intensive applications
 - “long” programs
 - Programs with many variables
- A definition-clear du-path represents a small function that can be tested by itself.
- If a du-path is not definition-clear, it should be tested for each defining node.
- **Note:** Applicability exists for Object-Oriented Code as well.

26

Program Slicing

Given a program, P , with program graph $G(P)$, and the set V of variables in program P . Nodes in the program graph are numbered and correspond to statement fragments.

Definition: The slice on the variable set V at statement fragment n , written $S(V, n)$, is the set of node numbers of all statement fragments in P prior to n that contribute to the values of variables in V at statement fragment n .

- This is actually a “backward slice”.

27

Backward Slice*

```
1. input(n)
2. i = 1
3. sum = 0
4. product = 0
5. while i <= n do
6.     sum = sum + i
7.     product = product * i
8.     i = i + 1
9. output(sum)
10. output(product)
```

What does this code do?

*Example based on Dr. Mary Jean Harrold's Work

28

Backward Slice

```
1. input(n)
2. i = 1
3. sum = 0
4. product = 0
5. while i <= n do
6.     sum = sum + i
7.     product = product * i
8.     i = i + 1
9. output(sum)
10. output(product)
```

What is the slice for
S(product,10)?

29

Backward Slice

1. **input**(n)
2. **i** = 1
3. **sum** = 0
4. **product** = 0
5. **while** **i** <= **n** **do**
6. **sum** = **sum** + **i**
7. **product** = **product** * **i**
8. **i** = **i** + 1
9. **output**(**sum**)
10. **output**(**product**)

What is the slice for
S(**product**,10)?

30

Backward Slice

1. **input**(n)
2. **i** = 1
3. **sum** = 0
4. **product** = 0
5. **while** **i** <= **n** **do**
6. **sum** = **sum** + **i**
7. **product** = **product** * **i**
8. **i** = **i** + 1
9. **output**(**sum**)
10. **output**(**product**)

What is the slice for S(**sum**,9)?

31

Backward Slice

1. **input**(n)
2. $i = 1$
3. $sum = 0$
4. $product = 0$
5. **while** $i \leq n$ **do**
6. $sum = sum + i$
7. $product = product * i$
8. $i = i + 1$
9. **output**(sum)
10. **output**(product)

What is the slice for $S(sum, 9)$?

32

Backward Slice

- | | |
|--------------------------------------|--------------------------------------|
| 1. input (n) | 1. input (n) |
| 2. $i = 1$ | 2. $i = 1$ |
| 3. $sum = 0$ | 3. $sum = 0$ |
| 4. $product = 0$ | 4. $product = 0$ |
| 5. while $i \leq n$ do | 5. while $i \leq n$ do |
| 6. $sum = sum + i$ | 6. $sum = sum + i$ |
| 7. $product = product * i$ | 7. $product = product * i$ |
| 8. $i = i + 1$ | 8. $i = i + 1$ |
| 9. output (sum) | 9. output (sum) |
| 10. output (product) | 10. output (product) |

What is the slice for
 $S(product, 10)$?

What is the slice for
 $S(sum, 9)$?

What is the difference between these two?

33

Backward Slice

```

1. input(n)
2. i = 1
3. sum = 0
4. product = 0
5. while i <= n do
6.     sum = sum + i
7.     product = product * i
8.     i = i + 1
9. output(sum)
10. output(product)

```

What is the slice for
S(product,10)?

```

1. input(n)
2. i = 1
3. sum = 0
4. product = 0
5. while i <= n do
6.     sum = sum + i
7.     product = product * i
8.     i = i + 1
9. output(sum)
10. output(product)

```

What is the slice for
S(sum,9)?

Are they both executable?

34

Forward Slice

If backward slicing looks at what could have effected a specific variable on a specific line, what do you think forward slicing is?

**A forward program slice are the statements that
may be affected by the value!**

35

Forward Slice

1. **input**(n)
2. $i = 1$
3. $sum = 0$
4. $product = 0$
5. **while** $i \leq n$ **do**
6. $sum = sum + i$
7. $product = product * i$
8. $i = i + 1$
9. **output**(sum)
10. **output**(product)

What is the *forward* slice for
S(sum,3)?

36

Forward Slice

1. **input**(n)
2. $i = 1$
3. $sum = 0$
4. $product = 0$
5. **while** $i \leq n$ **do**
6. $sum = sum + i$
7. $product = product * i$
8. $i = i + 1$
9. **output**(sum)
10. **output**(product)

What is the *forward* slice for
S(sum,3)?

37

Forward Slice

```

1. input(n)
2. i = 1
3. sum = 0
4. product = 0
5. while i <= n do
6.     sum = sum + i
7.     product = product * i
8.     i = i + 1
9. output(sum)
10. output(product)

```

Is this executable?

38

Lattice of Slices

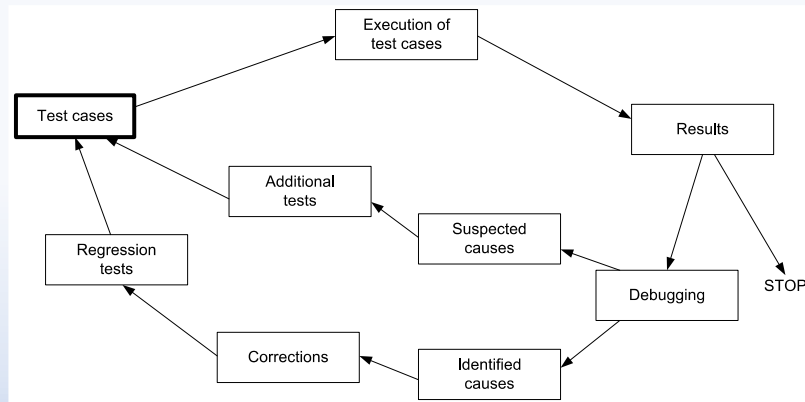
- Because a slice is a set of statement fragment numbers, we can find slices that are subsets of other slices.
- This allows us to “work backwards” from points in a program, presumably where a fault is suspected.
- The statements leading to the value of commission when it is output are an excellent example of this pattern.
- Some researchers propose that this is the way good programmers think when they debug code.

Speaking of Debugging...

39

Debugging Techniques

- Debugging always occurs as a consequence of testing.
- Debugging process begins with the execution of a test case.



40

Debugging Techniques

- The debugging process will always have one of two outcomes
 - The cause will be found and corrected, or
 - The cause will not be found in which case the person debugging may suspect a cause, design one or more test cases to help validate that suspicion, and work toward error correction in an iterative fashion.
- Debugging Techniques
 - Brute Force
 - Backtracking
 - Cause elimination
 - Automated debugging tools
 - People factor – a fresh viewpoint, unclouded by hours of frustration, from a colleague can do wonders

41

Debugging Techniques

- **Brute Force:** Most common and least efficient method for isolating the cause of an error
 - Hope to find a clue that can lead us to the cause of an error
 - Using memory dumps, run-time traces, loading program with output statements.
- **Backtracking:** a fairly common debugging approach that can be used successfully in small programs
 - Beginning at the site where a symptom has been uncovered, the source code is traced backward (manually) until the site of the cause is found.
 - As a number of source lines increase, the number of potential backwards paths may become unmanageably large.

42

Debugging Techniques

- **Cause Elimination:**
 - A “cause hypothesis” is devised and data is used to prove or disprove the hypothesis.
 - Alternatively, a list of all possible causes is developed, and tests are conducted to eliminate each cause in an attempt to isolate the bug
- **Interesting Quotes on Debugging:**
 - “The first step in fixing a broken program is getting it to fail repeatedly (on the simplest example possible).” – T. Duff
 - “Everyone knows that debugging is twice as hard as writing a program in the first place. So if you are as clever as you can be when you write it, how will you ever debug it?” – Brian Kernighan

43